Design for Everyone and Everywhere

Human-Computer Interaction

Tommaso Turchi — tommaso.turchi@unipi.it

06.11.2025

The Modern Interface Challenge

Today's digital products must work:

- For **everyone** — users with diverse abilities and needs
- **Everywhere** — smartphones, tablets, desktops, wearables
- **Consistently** — across teams, products, and time

> These aren't separate problems — they're interconnected design challenges

3 Pillars of Scalable Design

Accessibility Standards for designing interfaces that work for **all users**, including those with disabilities

Responsive Design Techniques for creating interfaces that adapt to **any device** or screen size

Design Systems Frameworks for maintaining **consistency** across products and teams

Together, they enable interfaces that scale effectively

Today's Journey

1. **Accessibility** — Understanding accessibility requirements and principles
2. **Responsive Design** — Adapting interfaces across devices
3. **Design Systems** — Systematizing design decisions

Goal

Build interfaces that are accessible and responsive by design, maintained consistently through systems

Accessibility

Designing for Everyone

Why Accessibility?

The reality:

- 15% of global population has some form of disability
- Temporary impairments affect everyone (broken arm, bright sunlight, noisy environment)
- Legal requirements in many countries (EU, US, others)

> **WCAG**
>
> Web Content Accessibility Guidelines — International standards (W3C) for making web content accessible

Current version: WCAG 2.2 (October 2023)

Structure: 4 principles → 13 guidelines → 78+ success criteria → 3 conformance levels

Good accessibility is good design

POUR Principles

Perceivable Information presentable in ways users can perceive (alt text for images, captions for video)

Operable UI components must be operable (keyboard access, sufficient time, no seizure triggers)

Understandable Information and operation must be clear (consistent navigation, error prevention)

Robust Content works with assistive technologies (semantic HTML, ARIA)

Conformance levels:

- **Level A**: Minimum (if not met, some users completely blocked)
- **Level AA**: Recommended baseline — **most legal requirements target this**
- **Level AAA**: Enhanced (not always achievable for all content)

Key Guidelines & Testing

Essential requirements (Level AA):

- **Color contrast**: 4.5:1 for text, 3:1 for UI components
- **Keyboard access**: All functionality available via keyboard, visible focus indicators
- **Alt text**: Meaningful descriptions for images (empty alt for decorative)
- **Semantic HTML**: Proper headings (h1→h2→h3), labels for form inputs, ARIA when needed
- **Consistency**: Navigation in same order, same functions labeled identically
- **Resizable text**: Interface works at 200% zoom without loss of functionality

Testing approach:

- **Automated** (30-40% of issues): WAVE, axe DevTools, Lighthouse
- **Manual** (essential): Keyboard-only navigation, screen readers (NVDA, JAWS, VoiceOver), zoom testing

Design with accessibility from the start — use semantic HTML, test with keyboard and screen readers

Responsive Design

Designing for Everywhere

The Multi-Device Reality

Users access interfaces on:

- Smartphones (320px - 428px width)
- Tablets (768px - 1024px width)
- Laptops (1280px - 1920px width)
- Desktop monitors (1920px+ width)
- Wearables, TVs, car displays...

One design must work across this massive range

What is Responsive Design?

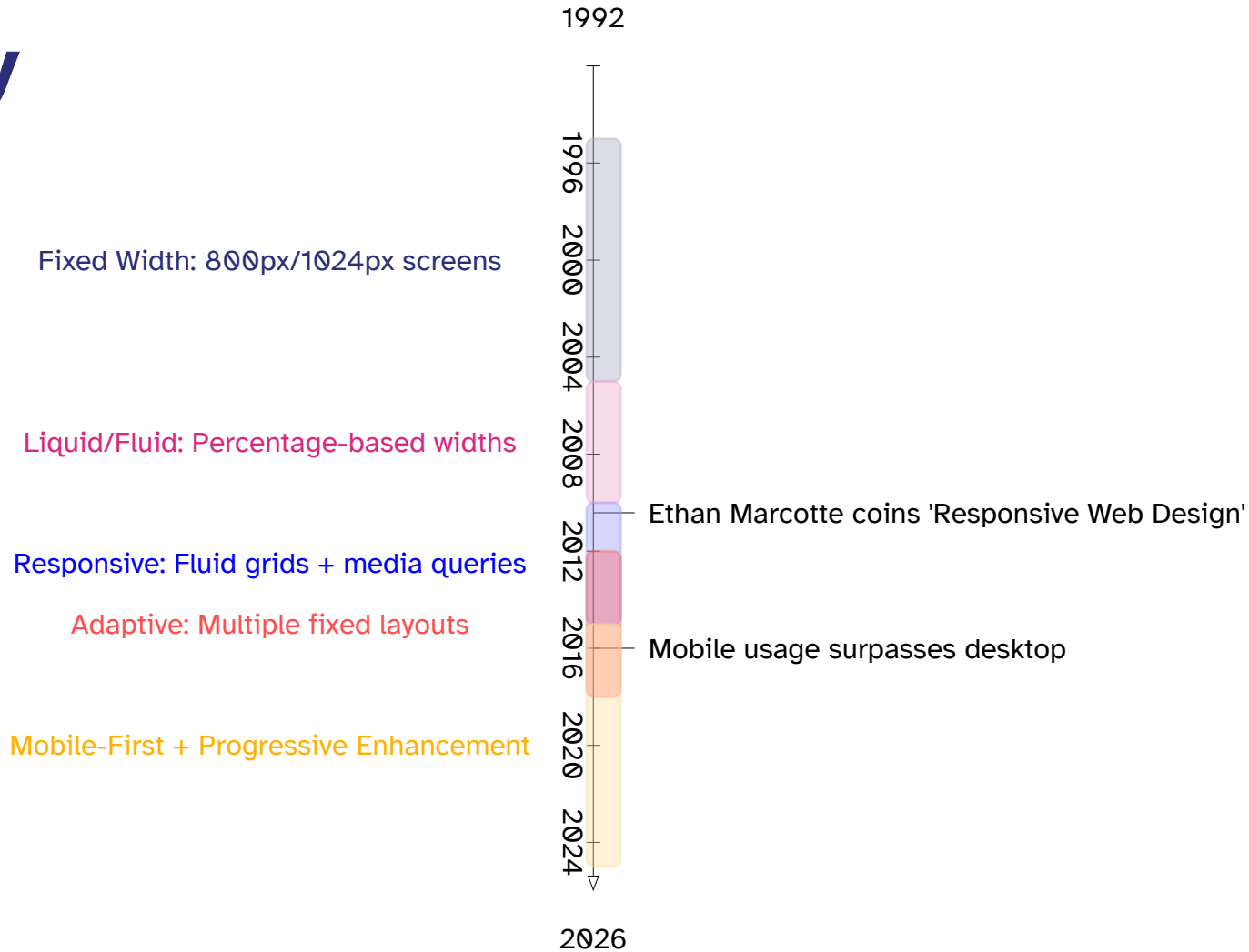
Definition

Design approach where interfaces adapt their layout, content, and functionality based on the device characteristics — especially screen size

Key techniques:

- Fluid grids (proportional rather than fixed widths)
- Flexible images and media
- Media queries (CSS rules based on device properties)
- Mobile-first thinking

History


1992
1996
2000
2004
2008
2012
2016
2020
2024
2026
Fixed Width: 800px/1024px screens
Liquid/Fluid: Percentage-based widths
Responsive: Fluid grids + media queries
Adaptive: Multiple fixed layouts
Mobile-First + Progressive Enhancement
Ethan Marcotte coins 'Responsive Web Design'
Mobile usage surpasses desktop

Mobile-First Approach

Why design for mobile first?

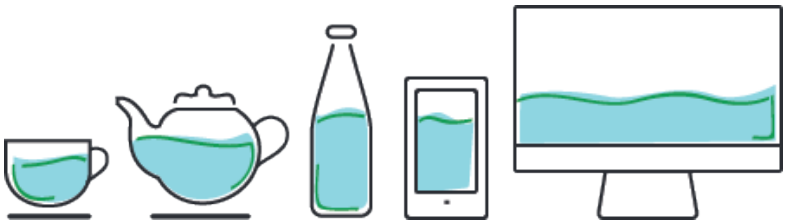
- Forces prioritization (limited space = focus on essentials)
- Easier to progressively enhance than gracefully degrade
- Mobile usage has surpassed desktop
- Performance benefits (mobile users often on slower connections)

> **Progressive Enhancement**
>
> Start with core functionality for all, add enhancements for capable devices

Breakpoints Strategy

Common breakpoints:

- 320px — Small phones (minimum)
- 375px — Standard phones
- 768px — Tablets portrait
- 1024px — Tablets landscape / small laptops
- 1280px — Laptops
- 1920px — Large desktops



> Don’t design for devices — design for content. Add breakpoints where your layout breaks

Better approach: Start small, expand viewport, add breakpoint when layout needs adjustment

Grid Systems: The Foundation

Grid System

Layout technique dividing the page into columns that elements can span

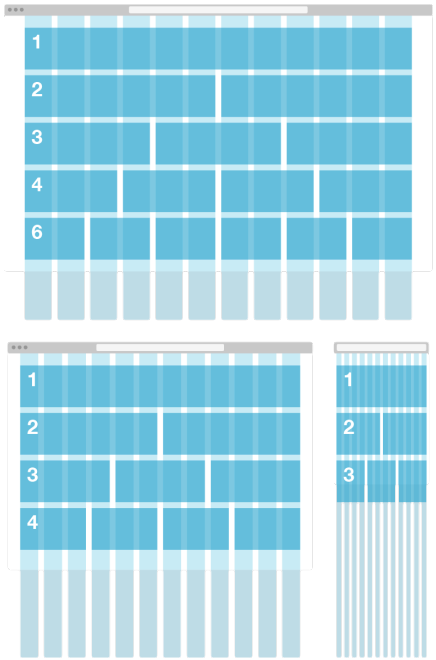
How it works:

- Page divided into equal-width columns (commonly 12)
- Elements can span 1 or more columns
- At different breakpoints, elements span different numbers of columns
- Gutters (spacing) between columns provide visual separation

Example:

- Desktop: Main content spans 8 columns, sidebar spans 4 columns
- Tablet: Main content spans 8 columns, sidebar spans 4 columns
- Mobile: Both span 12 columns (full width), stack vertically

Common systems: Bootstrap, Foundation, CSS Grid, Tailwind



Responsive Patterns: Tiny Tweaks

Simplest pattern

- Layout stays mostly the same across all screen sizes
- Only minor adjustments: font sizes, spacing, margins
- Single-column layout that works at all sizes

When to use:

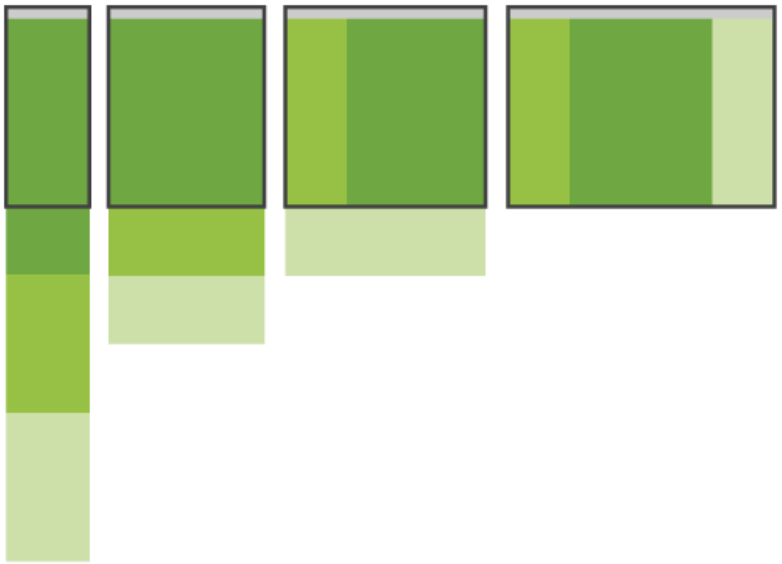
- Simple, linear content (articles, blogs, forms)
- Mobile-first designs that work well everywhere
- Single-purpose pages

Example: Simple articles, single-column blogs, linear forms

Responsive Patterns: Column Drop

Most common pattern

- Start with single-column layout (mobile)
- Add columns as screen width increases
- Columns “drop” below each other on narrow screens



When to use:

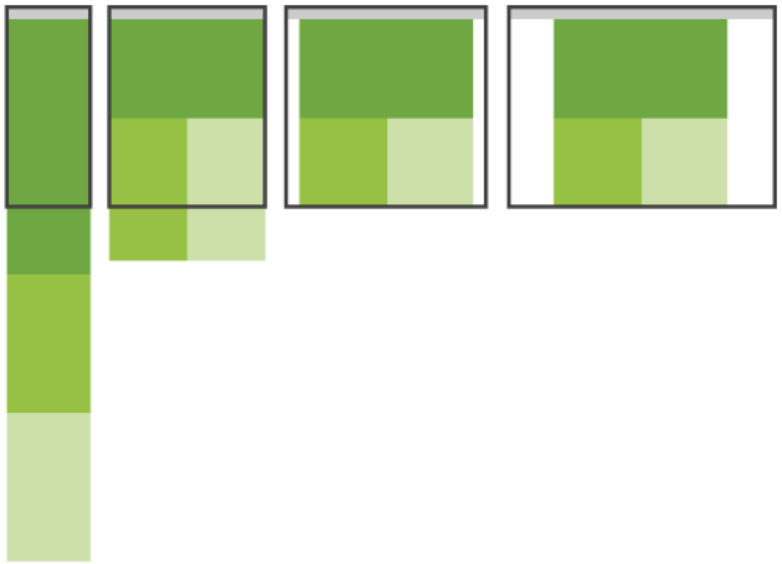
- Content-heavy sites
- Multi-column layouts on desktop
- Straightforward, predictable behavior

Example: News sites, blogs, documentation

Responsive Patterns: Mostly Fluid

Variation on column drop

- Grid becomes more fluid
- Margins increase on larger screens
- Maximum width prevents line length from becoming too long



When to use:

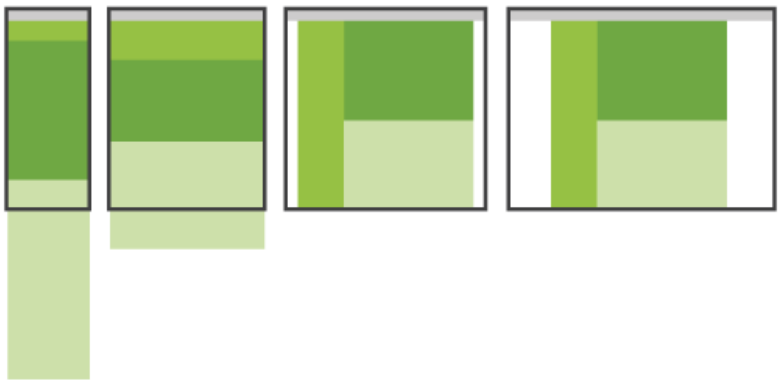
- Applications with defined content areas
- When readability on large screens matters
- Want consistent experience across sizes

Example: Email clients, productivity apps

Responsive Patterns: Layout Shifter

Most flexible, most complex

- Layout changes significantly at breakpoints
- Different arrangements on mobile vs tablet vs desktop
- Not just column dropping — fundamental reorganization



When to use:

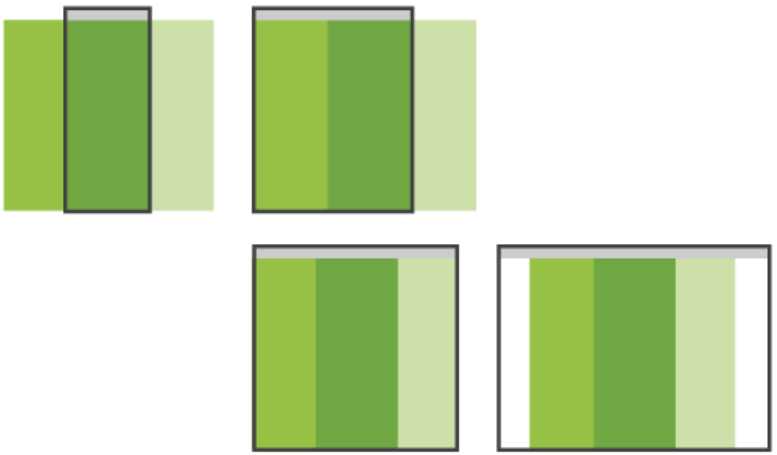
- When mobile and desktop experiences should differ significantly
- Dashboard-style interfaces
- Complex applications

Requires more design and development work

Responsive Patterns: Off Canvas

Hide content off-screen

- Navigation or secondary content hidden by default (mobile)
- Revealed via hamburger menu or gesture
- Always visible on larger screens



When to use:

- Apps with extensive navigation
- Secondary content that's not always needed
- Limited mobile screen space

Example: Gmail, admin dashboards, social media apps

Touch vs Mouse Interactions

Mobile introduces new interaction paradigms:

- No hover states (touch has no hover)
- Touch targets must be larger (min 44x44px recommended)
- Gestures (swipe, pinch, long-press)
- Thumb zones (bottom of screen easier to reach)

> Interface must work without hover — don't hide critical actions in hover-only menus

Responsive Images & Media

Problem: Sending large desktop images to mobile wastes bandwidth

Solutions:

- `srcset` attribute: Provide multiple image sizes, browser picks best
- `<picture>` element: Art direction (different crops for different sizes)
- `loading="lazy"`: Defer off-screen image loading
- Vector graphics (SVG) scale infinitely

Video considerations:

- Provide controls (WCAG requirement)
- Consider autoplay policies (many browsers block)
- Bandwidth considerations for mobile users

CSS Techniques

Modern layout tools:

- **Flexbox**: One-dimensional layouts (rows or columns) *flexboxfroggy.com*
- **CSS Grid**: Two-dimensional layouts (rows and columns) *cssgridgarden.com*
- **Media queries**: Apply styles based on screen size

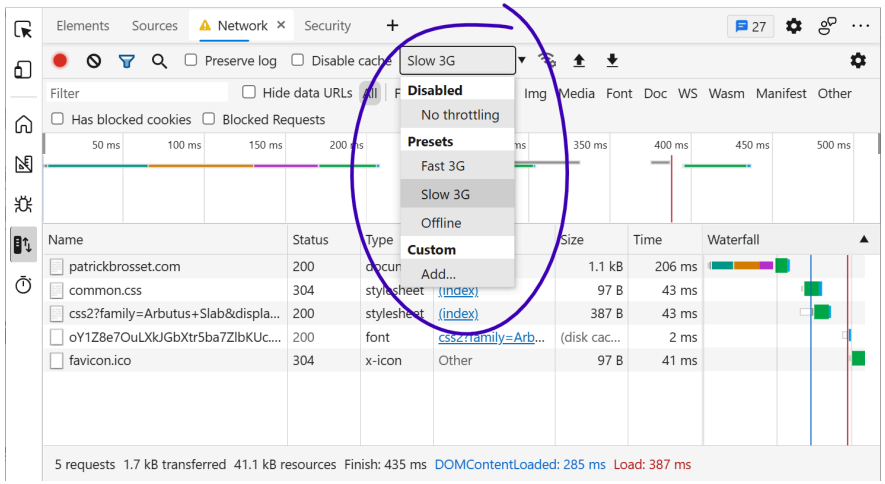
```
/* Mobile first */
.container { display: flex; flex-direction: column; }

/* Tablet and up */
@media (min-width: 768px) {
  .container { flex-direction: row; }
}
```

Performance Considerations

Mobile users face:

- Slower network connections (3G/4G vs WiFi)
- Limited data plans
- Less powerful processors
- Smaller battery capacity



Design implications:

- Minimize HTTP requests
- Optimize images aggressively
- Lazy load below-the-fold content
- Consider reduced motion preferences (WCAG)

Design Systems

Designing Consistently

The Consistency Problem

As products grow:

- Multiple designers make different decisions
- Components get reimplemented slightly differently
- Inconsistencies accumulate
- Accessibility and responsiveness vary
- Quality degrades over time

Without systems, maintaining quality at scale becomes impossible

What is a Design System?

Definition

A collection of reusable components, guided by clear standards, that can be assembled to build applications

A design system includes:

- Component library (buttons, forms, cards, navigation)
- Design tokens (colors, spacing, typography, shadows)
- Usage guidelines and documentation
- Design principles and patterns
- Code implementations

More than a component library — it's a shared language

Benefits of Design Systems

Consistency Same components behave identically everywhere
Efficiency Build once, reuse many times
Quality Accessibility and responsiveness built in
Scalability New team members onboard faster
Maintenance Fix bugs once, all instances benefit

Design systems are infrastructure investments that pay dividends over time

Atomic Design Methodology

Atoms Basic building blocks that can't be broken down further

- Example: Button, input field, label, icon

Molecules Simple combinations of atoms that function together

- Example: Search form = label + input + button

Organisms Complex UI components built from molecules and atoms

- Example: Site header = logo + navigation + search form

Templates Page-level layouts showing content structure

- Example: Homepage layout, article page layout

Pages Specific instances of templates with real content

- Example: Actual homepage with real text and images

Atomic Design Methodology

Atoms Basic building blocks that can't be broken down further

- Example: Button, input field, label, icon

Molecules Simple combinations of atoms that function together

- Example: Search form = label + input + button

Organisms Complex UI components built from molecules and atoms

- Example: Site header = logo + navigation + search form

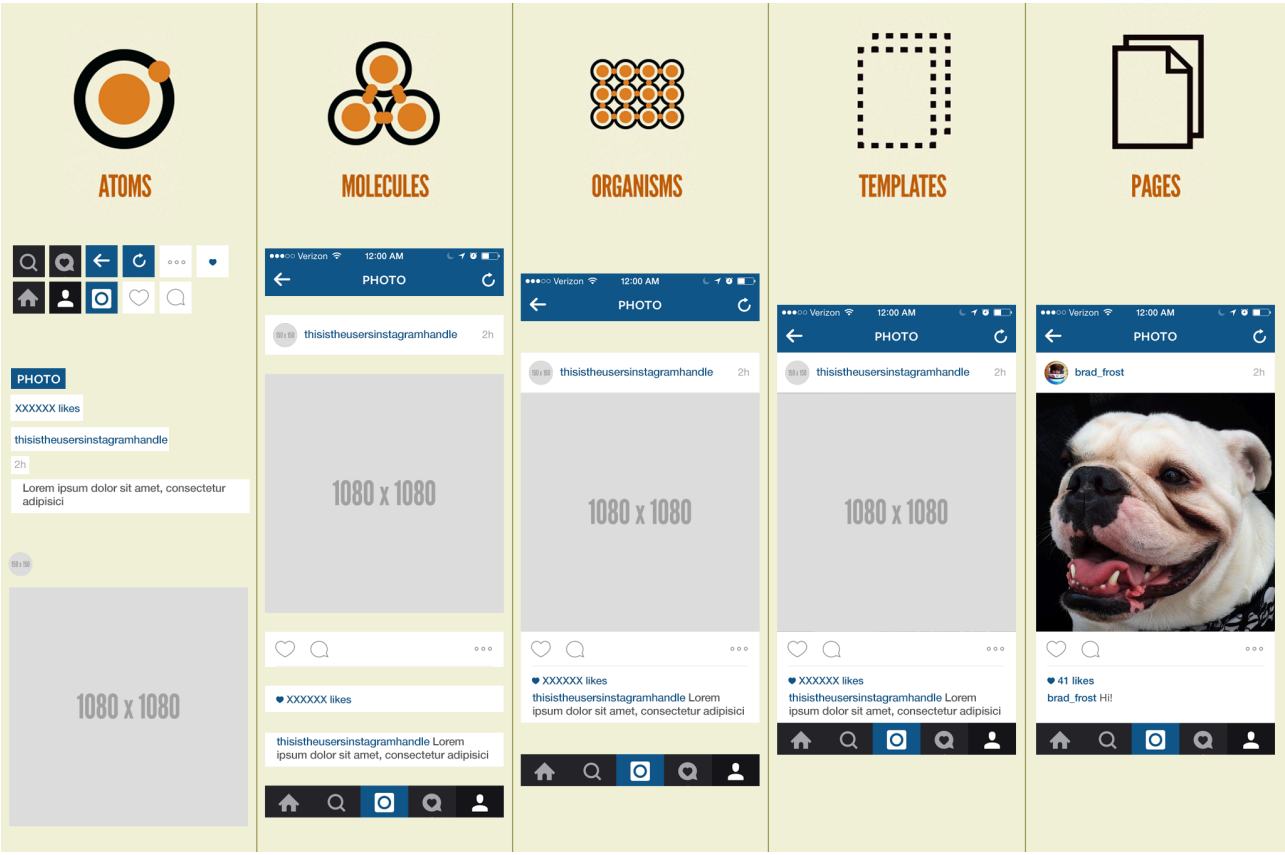
Templates Page-level layouts showing content structure

- Example: Homepage layout, article page layout

Pages Specific instances of templates with real content

- Example: Actual homepage with real text and images

Atomic Design: Example



Design Tokens

Design Tokens

Named design decisions stored as data — the foundation of your system

Examples:

```
color-primary-500: #3B82F6
spacing-md: 16px
font-size-lg: 18px
shadow-md: 0 4px 6px rgba(0,0,0,0.1)
border-radius-sm: 4px
```

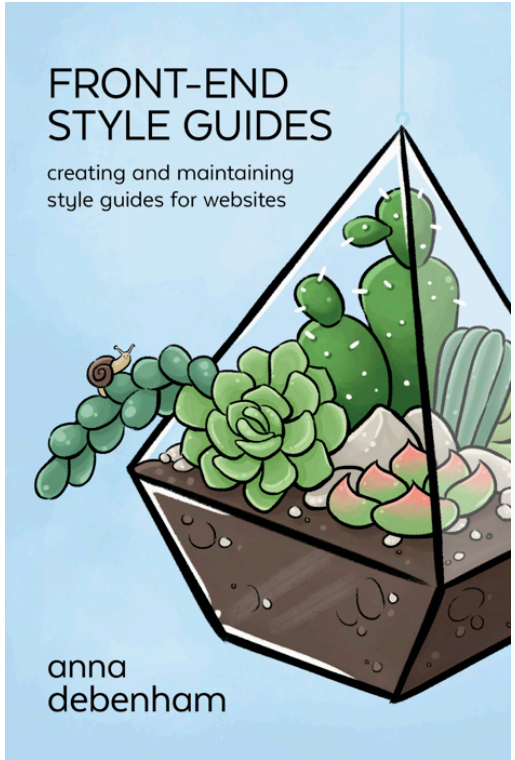
Why tokens matter:

- Single source of truth for design decisions
- Change once, update everywhere
- Platform-agnostic (generate CSS, iOS, Android code)
- Semantic naming improves maintainability

Front-End Style Guides

What to document:

- Visual design patterns (colors, typography, spacing)
- UI components and their variations
- Code examples showing implementation
- Usage guidelines: when to use, when not to use
- Accessibility requirements for each component
- Do’s and don’ts with visual examples



Why documentation matters:

- Onboards new team members quickly
- Reduces repeated questions
- Maintains consistency across teams
- Creates shared understanding between designers and developers

Living Documentation

Best practices:

- Keep documentation alongside code
- Show interactive component examples
- Include real code snippets developers can copy
- Document component states (default, hover, disabled, error)
- Explain the “why” behind decisions, not just the “what”
- Make it searchable

Tools:

- Storybook — Interactive component explorer *storybook.js.org*
- Figma — Design documentation and hand-off
- Custom documentation sites

Example: Material Design

m3.material.io

Google's Material Design demonstrates:

- Comprehensive documentation
- Interactive examples
- Clear usage guidelines
- Accessibility guidance built in
- Cross-platform implementation

Other Notable Design Systems

Apple Human Interface Guidelines Platform-specific (iOS, macOS), emphasis on consistency within Apple ecosystem

Carbon (IBM) Enterprise-focused, accessible, comprehensive data visualization components

Polaris (Shopify) E-commerce focused, merchant experience patterns

Ant Design Enterprise applications, extensive component library

Atlassian Design System Collaboration tools, flexible theming

Study these to understand different approaches and trade-offs

Key Takeaways

- **WCAG isn't optional** — Accessibility is a legal requirement and design imperative; aim for Level AA complianc
- **Design mobile-first** — Start with constraints, progressively enhance; responsive design and accessibility naturally overlap
- **Build systems from atoms up** — Atomic Design provides a mental model for creating scalable, reusable component libraries
- **Documentation is critical** — Style guides turn component libraries into design systems by explaining the "why" and "how"
- **Learn from existing systems** — Study Material Design, Apple HIG, and others to understand best practices

Resources

WCAG:

- WCAG 2.2 Guidelines: https://www.w3.org/WAI/WCAG22/quickref/
- WebAIM: https://webaim.org/

Responsive Design:

- Responsive Patterns: https://developers.google.com/web/fundamentals/design-and-ux/responsive/patterns
- This Is Responsive: https://bradfrost.github.io/this-is-responsive/

Design Systems:

- Atomic Design: https://atomicdesign.bradfrost.com/
- Material Design: https://m3.material.io/
- Front-end Style Guides: http://styleguides.io/
- Design Systems Repo: https://designsystemsrepo.com/

Questions?